

Embedded Software Engineering

Simon Künzli, Tobias Welti
HS 2025

Agenda HS25

Vorlesung (welo)

Vorlesung (kuex)

Übungen / Praktika

Stand:
27.08.25

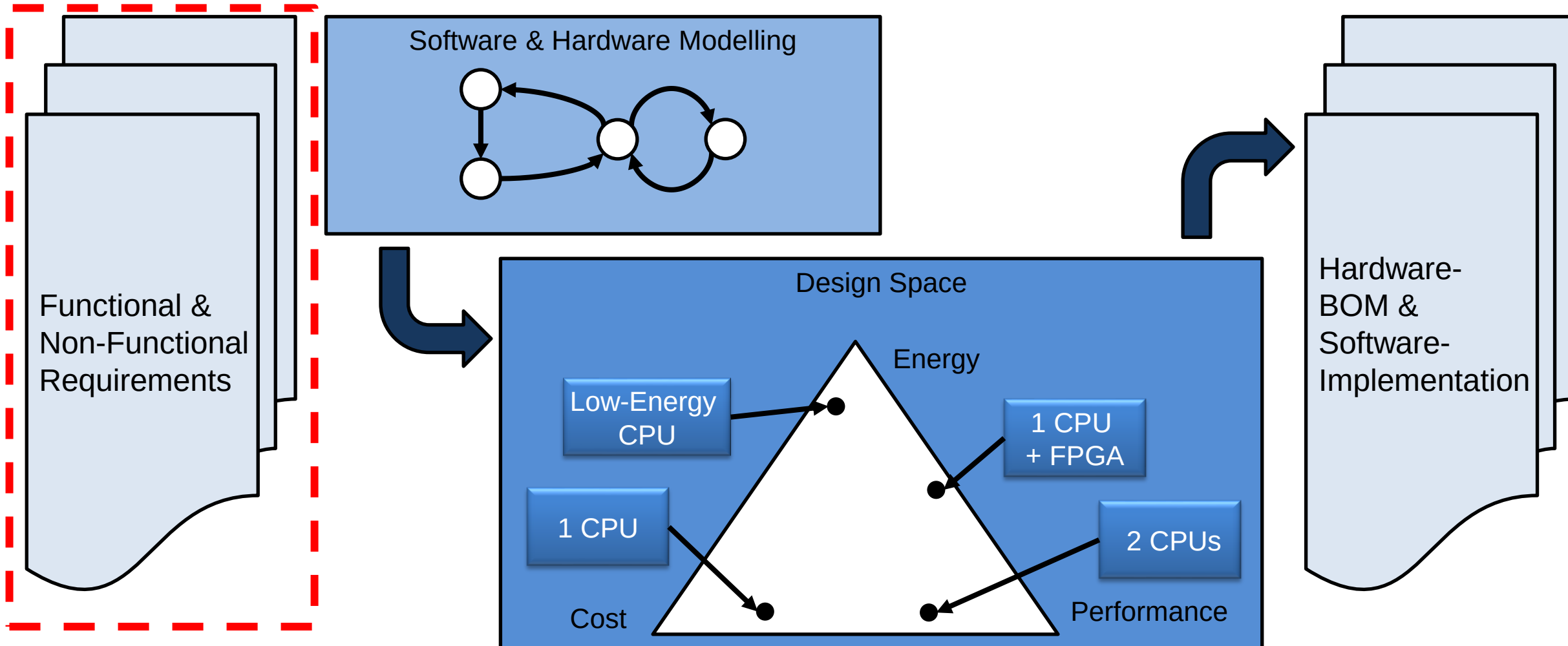
wk #	Do, 10:00 - 11:35	Do, 08:00 – 09:35 / Do 12:00 – 13:35	
01 15.09.	Modulorganisation & Einführung Embedded Systems	No Lab	
02 22.09.	SW-Paradigmen für Embedded Systems	No Lab	P1: First steps U96
03 29.09.	Requirements für Embedded Systems	P1: First steps U96	P1: Finish Lab
04 06.10.	Software-Entwicklung für ES (Modellierung, SW-Specs)	P1: Finish Lab	U1: Requirements
05 13.10.	Entwicklungsprozesse / CI/CD	U1: Requirements	U2: Modeling
06 20.10.	Non-functional Requirements	U2: Modeling	U3: Implementation & Test
07 27.10.	Visite Fertigung Siemens	Visite Fertigung Siemens	
08 03.11.	Fokus "Energie« / andere Prozessoren?	U3: Implementation & Test	P2: Energy Consumption
09 10.11.	Fokus "Performance"	P2: Energy Consumption	P3: AES in SW, incl. Measurement on R5
10 17.11.	Einführung FPGA	P3: AES in SW, incl. Measurement on R5	P3: Finish Lab
11 24.11.	Einführung Design Space Exploration (DSE)	P3: Finish Lab	P4: AES in FPGA and HW-Accelerator
12 01.12.	DSE: Kostenfunktionen und Suche im Entwurfsraum	P4: AES in FPGA and HW-Accelerator	P4: Finish Lab
13 08.12.	Einführung RTOS	P4: Finish Lab	P5: DSE
14 15.12.	Dual-Prozessor-Systeme mit Uniform Memory Access	P5: DSE	No Lab

After today's lesson, you will be able to:

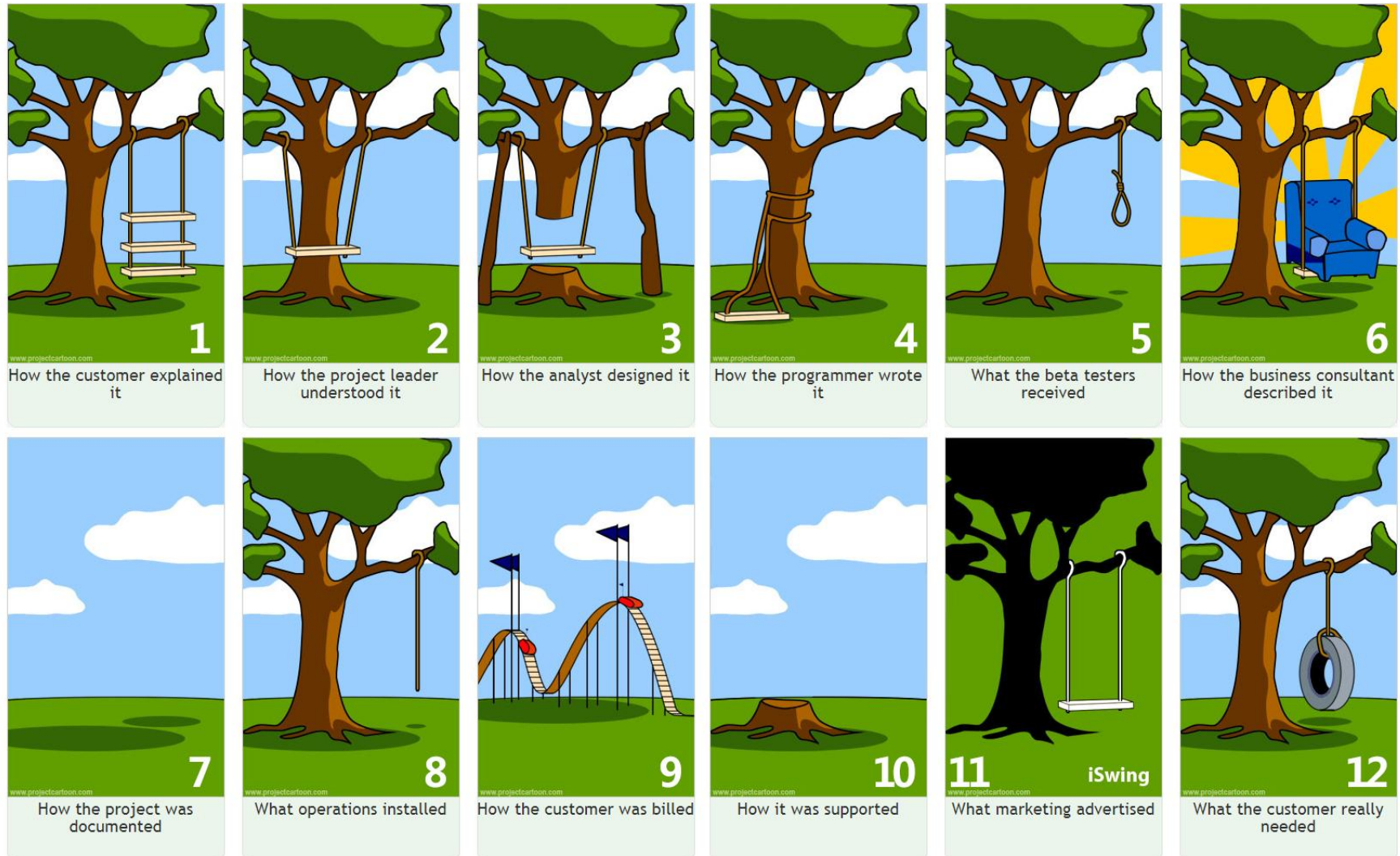
- write requirements for an Embedded System
- read and understand requirements for Embedded Systems
- decide whether a requirement is a MUST, SHOULD, or OPTIONAL requirement
- judge the quality of requirements and explain what's missing for a specific requirement
- explain the benefits of modelling methods for Embedded System's software
- explain the principles and rules for simple UML State Machines
- model an Embedded System's functionality using simple UML State Machines

ESE – Advance Organizer

Embedded Systems Engineering



Requirements – Motivation



Source: <https://pmac-agpc.ca/sites/default/files/Tree.jpg>

Requirements – Definition

- **Wikipedia:** «In product development and process optimization, a **requirement** is a singular documented physical or functional need that a particular design, product or process aims to satisfy. [..]»
- **In our context: “A requirement describes a behavior or property that the Embedded System shall achieve or implement.”**
- **A requirement can describe the functional behavior but also cover non-functional aspects of the product to be developed.**
 - Example for functional requirement: “The ES shall beep when connected to a battery”
 - Example for non-functional requirement: “The ES shall weigh less than 100g.”

Characteristics of a «good» Requirement?



Requirements – Characteristics

- A requirement needs to meet several criteria to be considered a “good requirement” (see also [1]).

- Good requirements should, e.g., have the following characteristics:
 - Unambiguous
 - Testable (verifiable)
 - Clear (concise, terse, simple, precise)
 - Correct
 - Understandable
 - Feasible (realistic, possible)
 - Independent
 - Atomic
 - Necessary
 - Implementation-free (abstract)

[1] Hull, Elizabeth, Kenneth Jackson, and Jeremy Dick. Requirements Engineering, London: Springer, 2005.

Requirements – Characteristics (II)

■ **Unambiguous**

- There should be only one way to interpret the requirement.
 - “The system shall allow the user to only view one menu entry.”
→ What exactly does this mean?
-
- How could we re-write the requirement above to be un-ambiguous?

Requirements – Characteristics (III)

■ Testable (Verifiable)

- A tester should be able to test whether the requirement is met and whether the implementation fulfills the requirement correctly.
 - Some words may turn a requirement un-testable: e.g., **secure, effective, efficient, flexible, maintainable, robust, reliable, user-friendly, good-looking, easy-to-use, simple**
 - Some other aspects are problematic, too: e.g., **quickly, securely, with meaningful effort, in a timely manner**
 - Even worse: **etc., and/or, TBD, «...»**
- “The system shall be easy to extend with new functionality.”
 - How can we decide on whether the requirement has been met?

Requirements – Characteristics (IV)

■ Independent

- To understand a single requirement, there should not be a need to know any other requirement.
- Example:
 - REQ1: *"The list of available flights shall include flight numbers, departure time, and arrival time for every leg of a flight."*
 - REQ2: *"It should be sorted by price."*

REQ2 cannot be understood without REQ1

→ How can we re-write REQ2 to be independent of REQ1?

Requirements – Characteristics

- A requirement needs to meet several criteria to be considered a “good requirement” (see also [1]).
- Good requirements should, e.g., have the following characteristics:
 - Unambiguous
 - Testable (verifiable)
 - Clear (concise, terse, simple, precise)
 - Correct
 - Understandable
 - Feasible (realistic, possible)
 - Independent
 - Atomic
 - Necessary
 - Implementation-free (abstract)

[1] Hull, Elizabeth, Kenneth Jackson, and Jeremy Dick. Requirements Engineering, London: Springer, 2005.

Requirements – Systems

- Besides the criteria for individual requirements, the following three criteria apply to the whole set of requirements for a system.
- The set should be:
 - Consistent (no contradictions)
 - Nonredundant
 - Complete (no missing information)
- Example: The requirements for a Siemens smoke detector contain over 700 individual requirements and cover many aspects: mechanical properties, packaging, standard compliance, process compliance, product approval, performance, reliability, design, etc.

Requirements – Writing

- **Often used terminology described in RFC2119**
 - Although the RFC 2119 defines the terminology to be used with W3C RFCs («standards for the internet»), the terms nicely fit the needs for describing requirements

Network Working Group
Request for Comments: 2119
BCP: 14
Category: Best Current Practice

S. Bradner
Harvard University
March 1997

Key words for use in RFCs to Indicate Requirement Levels

Status of this Memo

This document specifies an Internet Best Current Practices for the Internet Community, and requests discussion and suggestions for improvements. Distribution of this memo is unlimited.

Abstract

In many standards track documents several words are used to signify the requirements in the specification. These words are often capitalized. This document defines these words as they should be interpreted in IETF documents. Authors who follow these guidelines should incorporate this phrase near the beginning of their document:

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC 2119](#).

Note that the force of these words is modified by the requirement level of the document in which they are used.

- 1. MUST** This word, or the terms "REQUIRED" or "SHALL", mean that the definition is an absolute requirement of the specification.
- 2. MUST NOT** This phrase, or the phrase "SHALL NOT", mean that the definition is an absolute prohibition of the specification.
- 3. SHOULD** This word, or the adjective "RECOMMENDED", mean that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.
- 4. SHOULD NOT** This phrase, or the phrase "NOT RECOMMENDED" mean that there may exist valid reasons in particular circumstances when the particular behavior is acceptable or even useful, but the full implications should be understood and the case carefully weighed before implementing any behavior described with this label.

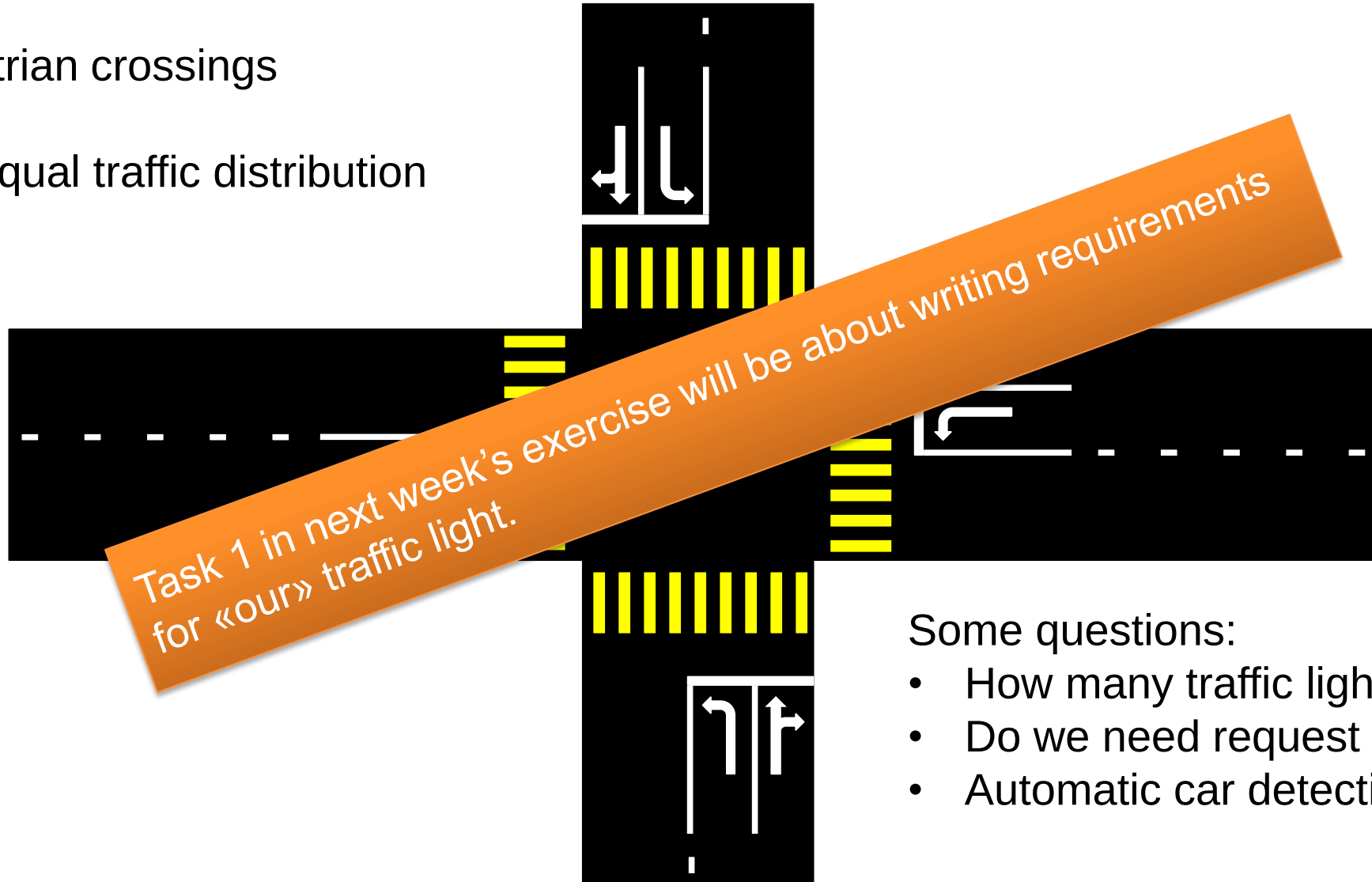
Requirements – RFC 2119

1. **MUST** - This word, or the terms "**REQUIRED**" or "**SHALL**", mean that the definition is an absolute requirement of the specification.
2. **MUST NOT** - This phrase, or the phrase "**SHALL NOT**", mean that the definition is an absolute prohibition of the specification.
3. **SHOULD** - This word, or the adjective "**RECOMMENDED**", mean that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.
4. **SHOULD NOT** - This phrase, or the phrase "**NOT RECOMMENDED**" mean that there may exist valid reasons in particular circumstances when the particular behavior is acceptable or even useful, but the full implications should be understood and the case carefully weighed before implementing any behavior described with this label.
5. **MAY** - This word, or the adjective "**OPTIONAL**", mean that an item is truly optional. One vendor may choose to include the item because a particular marketplace requires it or because the vendor feels that it enhances the product while another vendor may omit the same item. [..]

Source: <https://www.rfc-editor.org/rfc/pdf/rfc2119.txt.pdf>

Coming back to Example 1...

- Pedestrian crossings
- Cars
- (Un-)Equal traffic distribution



Some questions:

- How many traffic lights?
- Do we need request buttons?
- Automatic car detection?

Requirements - Examples

- «The traffic light should support displaying three colors.»
- «The traffic light control system shall react to the current traffic situation»
- «The traffic light control must count the number of cars waiting.»
- «The traffic light control system must count the number of cars waiting per lane.»
- «The traffic light control system must comply with ‘Wegleitung Lichtsignalanlagen V1.2 (Dez 2020) Dokumentennummer: 601.06.40’»

Source: https://www.zh.ch/content/dam/zhweb/bilder-dokumente/themen/planen-bauen/tiefbau/strassenanlagen/verkehrstechnik/lichtsignalanlagen/dokumente/601.06.40_wegleitung_lsa.pdf

Requirements – Priorities

- **Often requirements have different priorities for the customers**
 - **In other words, not all requirements are equally important to the customer**
 - Example Kitchen:
 - REQ1: “There must be an oven in the kitchen.”
 - REQ2: “In the kitchen, a hook should be available to hang towels close to the sink.”
 - **Introducing the “MoSCoW” principle**
 - ... must have ...
 - ... should have ...
 - ... could have ...
 - ... won't have ...
- Remark the last bullet point where a negative requirement is formulated

Requirements – Priorities

■ **Alternative I: Priorities by numbers:**

- “0”: Requirement is a must. Without meeting this requirement, the product might not be sold in the market at all
- “1”: Requirement is a must. Without meeting this requirement, a significant product feature will be missing, or a USP will be missing.
- “2”: Requirement is essential, but it’s possible not to implement it without harming the product’s primary purpose.
- “3”: Requirement is a nice-to-have feature that the customer would appreciate. This feature should cause no extra cost during development.

■ **Alternative II: Impact-Effort-Matrix**

- Evaluate and rank requirements based on their potential impact and the effort required to implement

Requirements – Engineering

- In industries, many hours are spent on the process of requirements engineering

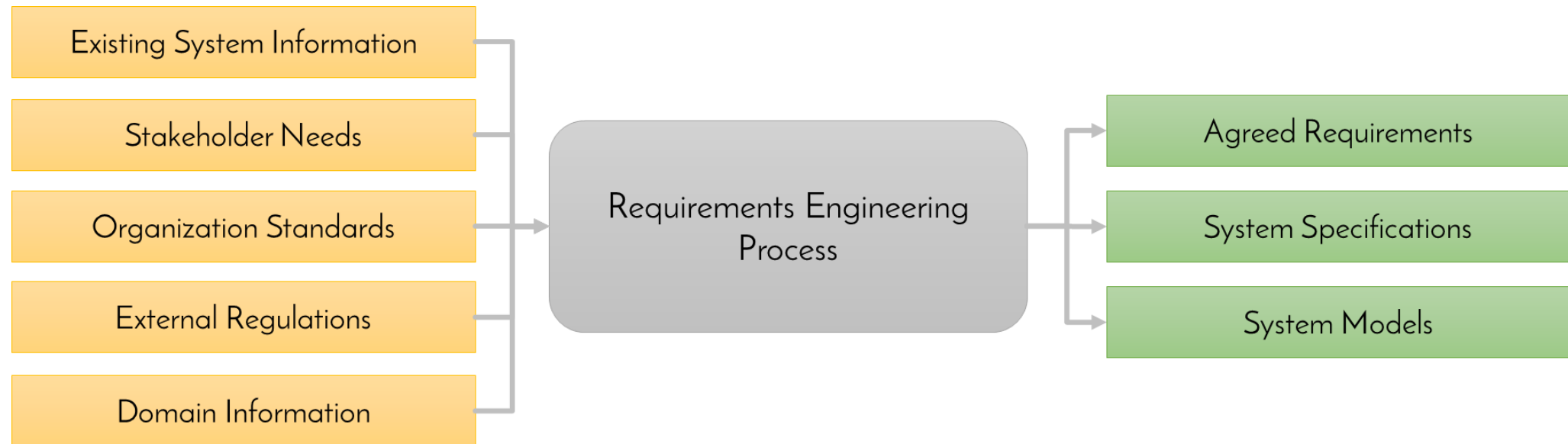
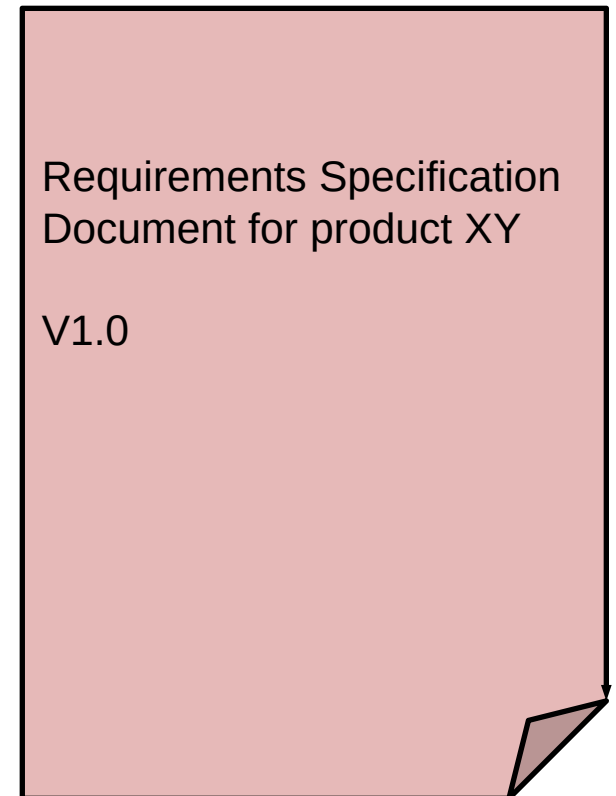
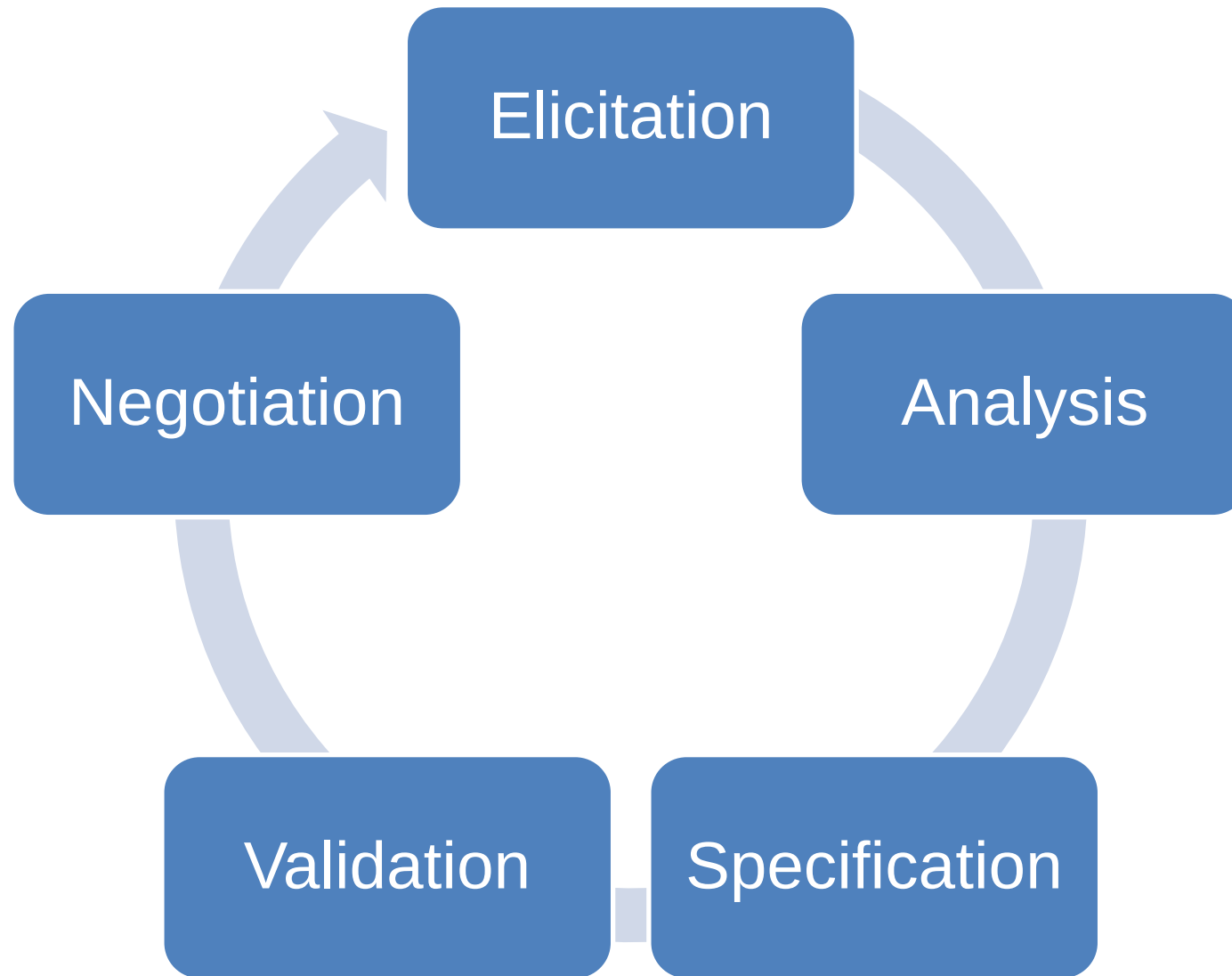


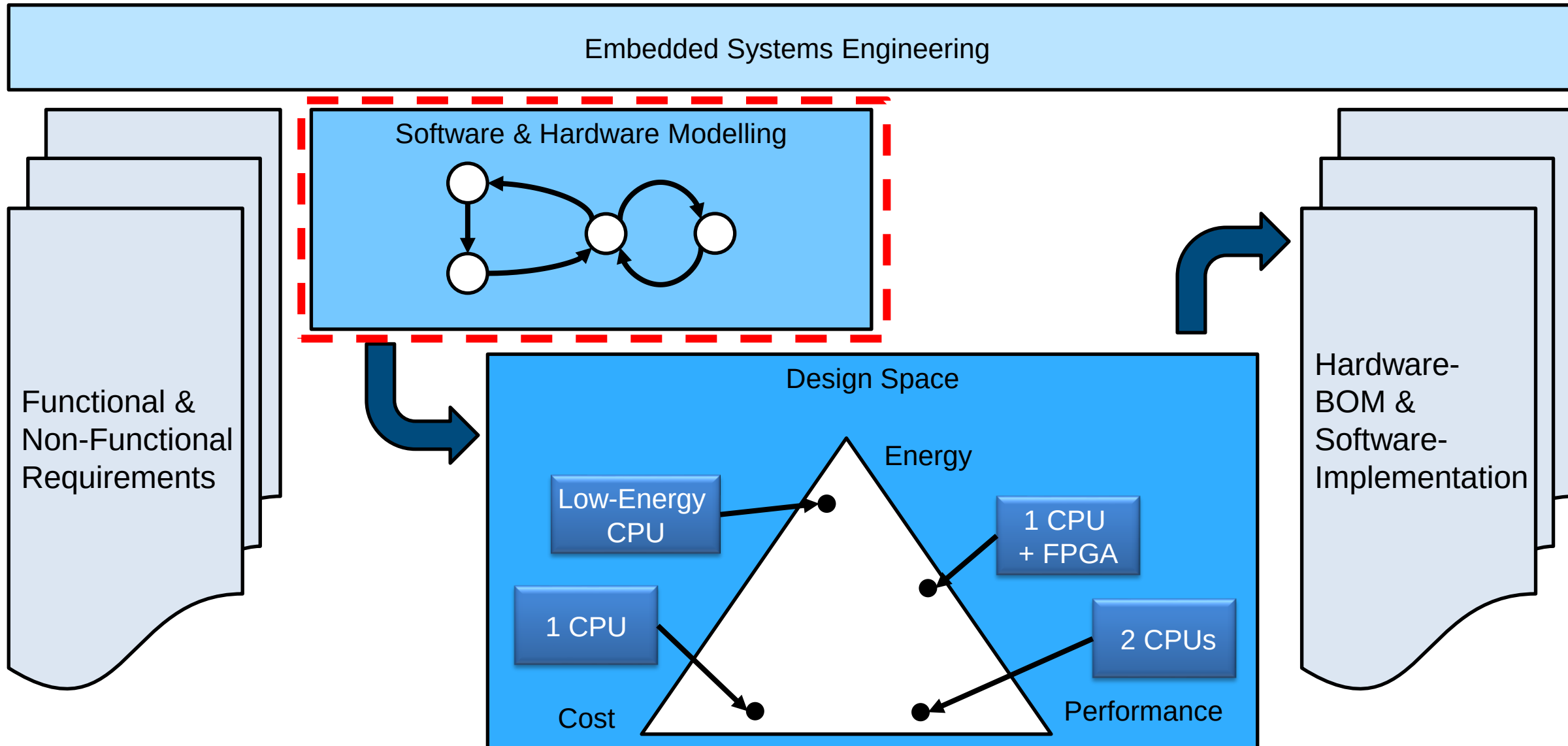
Illustration: The overall requirements engineering process

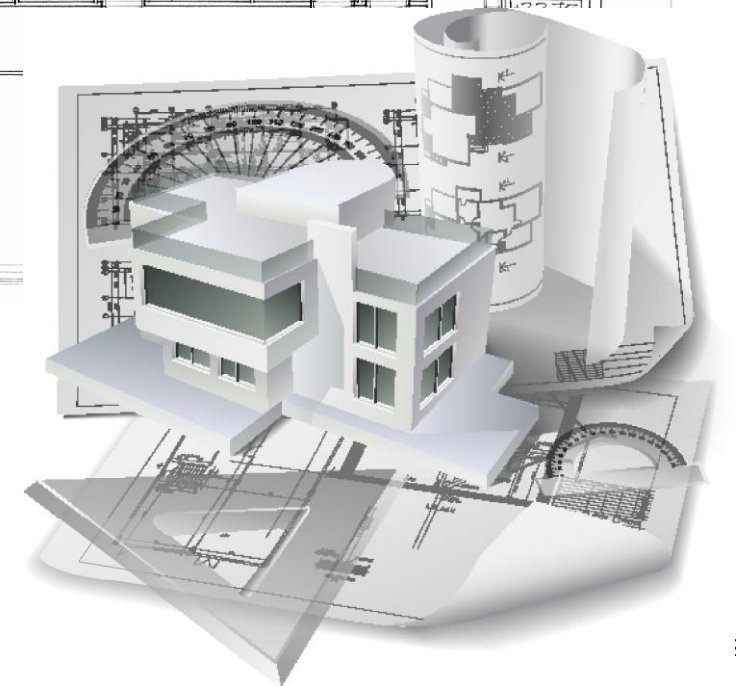
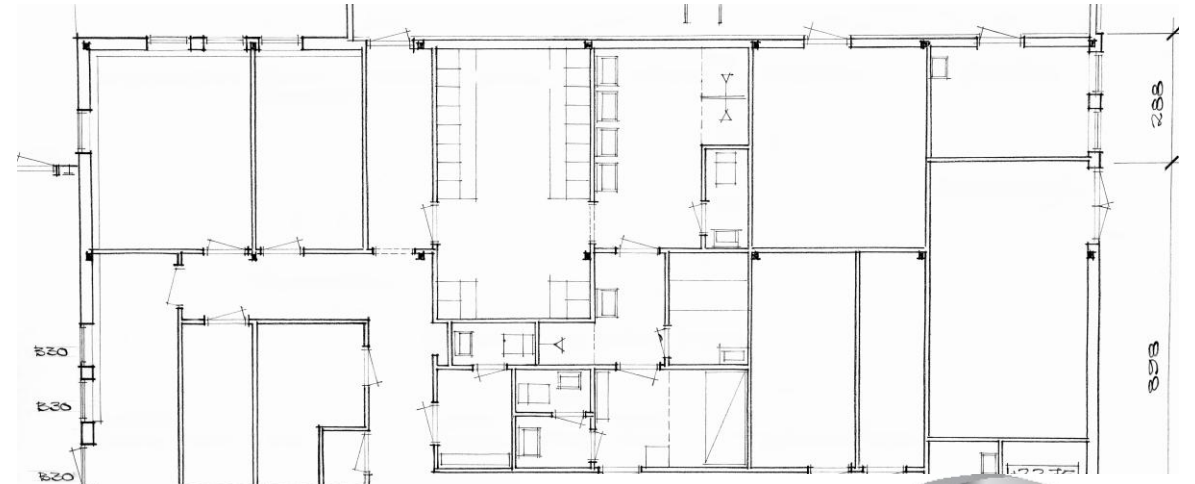
Source: <https://www.inflectra.com/Ideas/Whitepaper/Principles-of-Requirements-Engineering.aspx>

Requirements – Engineering (II)



ESE – Advance Organizer





Let's start with SW Modelling...

From Requirements to SW Sub-System Model:

- 1) Understand all Requirements**
- 2) Classify Requirements**
- 3) Partition System in Sub-Systems, where possible (e.g. Mech, HW, SW)**
- 4) Refine & Filter Requirements**
- 5) Create Sub-System Specification (What will my Sub-System do? How will my Sub-System fulfill all agreed Requirements?)**
- 6) Model the Functional Behavior of the Sub-System(s)**

Modelling Embedded Systems

- **ES are reactive systems that respond or react to signals from their environment:**
 - Smoke detector triggers an alarm in case it detects a fire (within seconds)
 - ABS reduces braking pressure in case it detects blocking of wheels (within microseconds)
 - Autonomous car stops in case it detects an obstacle in its way (within milliseconds)

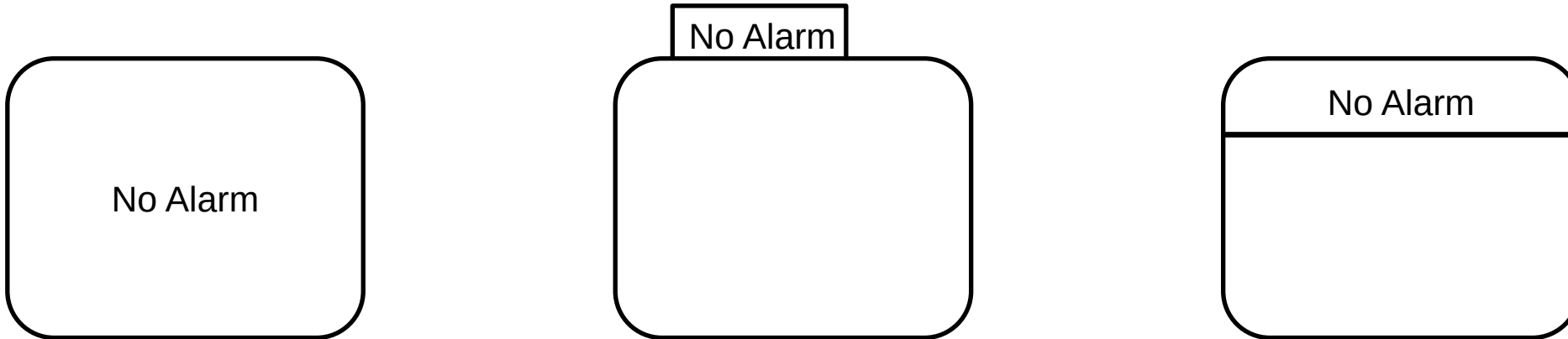
- **In order to properly design ES Software, i.e. to fulfill requirements, we need mechanisms for modeling such systems**

- **During the class this and next week, we will learn about the UML state machine modeling technique for ES Software in more detail:**
 - Simple UML State Machines (this week)
 - Extensions of UML State Machines (next week)
 - Verification of State Machines (next week)
 - Model-based Code Generation (next week)

Recap: UML State Machines

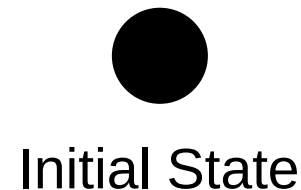
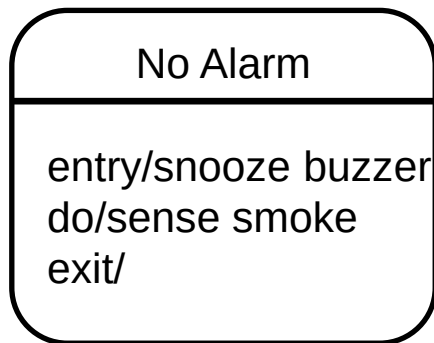
- **In UML State Machines, we know 4 types of elements:**
 - States
 - Transitions
 - Events
 - Actions

- States are used to represent the modes of operation of a system.
- States are represented as rounded rectangles containing the *name* of the state. All representations below denote the same state:



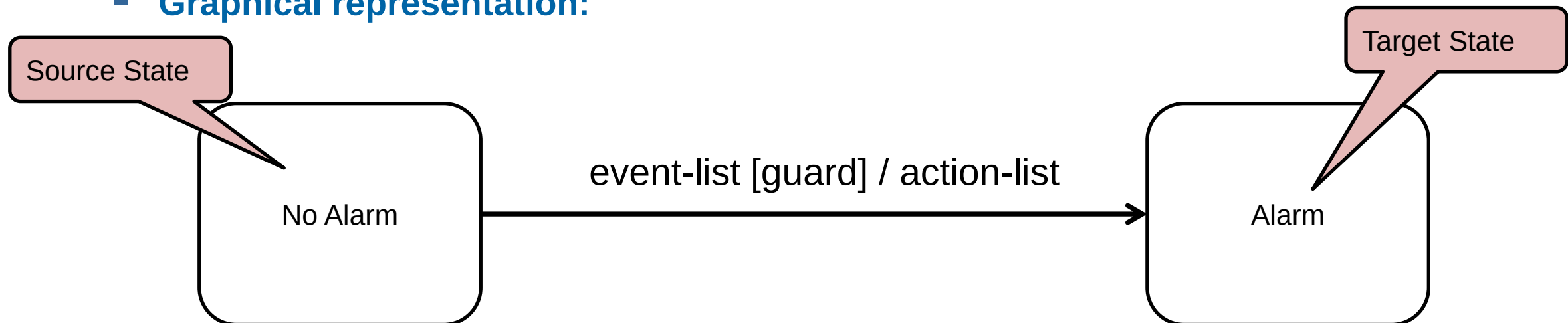
- In contrast to FSM, in UML State Machines, there are different types of states: *simple states, composite states*
 - Composite state types are used for modelling concurrency and hierarchy
 - *There are submachine states, too, but we will not cover them here.*

- States may be dynamic in UML State Machine notation, i.e. they can perform some action while they are *active states*
- Actions in states can occur at three different phases in the state:
 - entry: The associated action is performed when the state becomes active
 - exit: The associated action is performed right before the state becomes inactive
 - do: The associated action is performed as long as the state is active
- Graphical notation:



Transitions

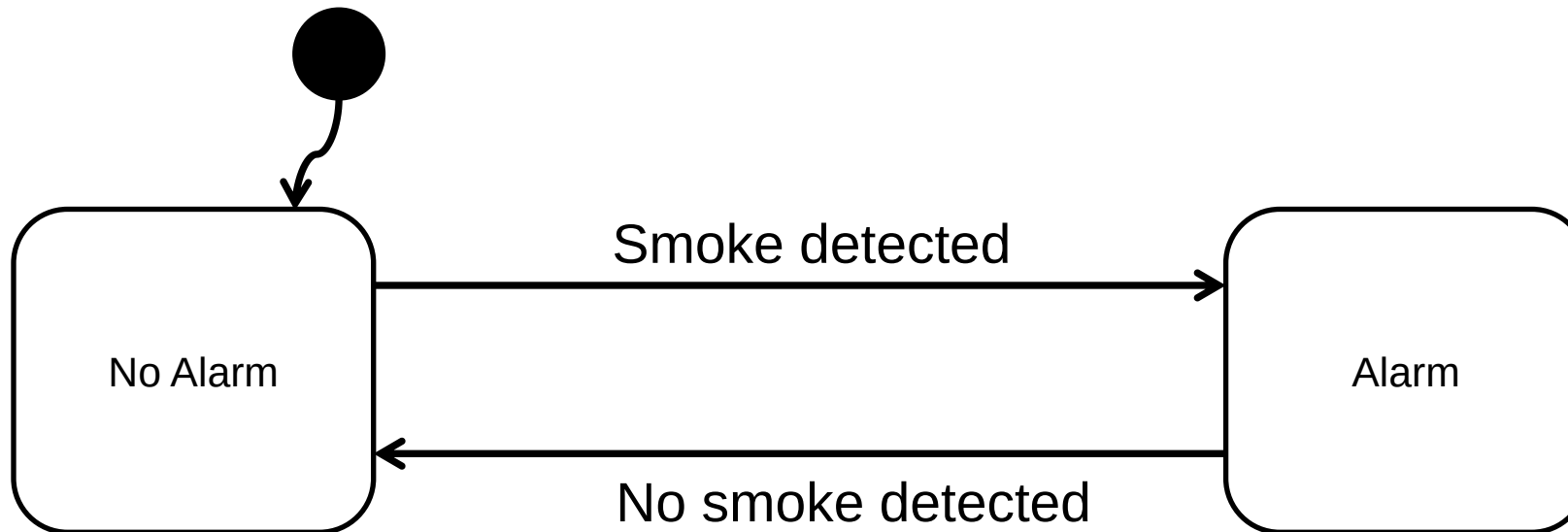
- Transitions denote changes in operational modes, i.e. a change of state
- Self-transitions are also allowed (same source state as target state)
- Transitions may be labeled with:
 - Event-list: A list containing the events that trigger the transition.
 - Guard: Condition that can control whether a transition can be triggered or not
 - Action-list: A list containing the actions that are executed when the transition is triggered
- Graphical representation:



- **Events are internal or external interactions that cause a transition to be triggered, therefore a change of behavior of the system**
- **If an event occurs, there are 2 possible scenarios:**
 1. The system changes its behavior (i.e. state) if the system is in a state that can “accept” an event
 2. The system doesn’t change its behavior if the system cannot handle the event
- **In UML State Machines, the following event types exist:**
 - **Signal event:** asynchronous notification by another object
 - **Call event:** synchronous event, represents dispatching of an operation (“event_name()”)
 - **Change event:** event when some condition is satisfied (“when(condition)”)
 - **Time event:** Example “after(60s)”. Time-outs are started when state becomes active and cancelled when the state becomes inactive. Only one time-out can be used per state.

Recap: UML State Machines

- UML State Machines can be represented in graphical form
- Example: Smoke Detector (simplified 😊)



Programming State Machines

■ Approach 1: Two levels of switch-case statements:

- Level 1: Switch on state
- Level 2: Switch on possible inputs

```
switch (current_state) :  
    case q0:                // «no alarm»  
        switch(input) :  
            case e1:         // «smoke detected»  
                current_state = q1;  
                break;  
            case e0:         // «no smoke dtd»  
                ...
```

Pro's:

- Approach is straight forward and easy to understand

Con's:

- Might grow quickly, not easy for maintaining code
- Bad for readability

Programming State Machines

■ Approach 2: Table-Based Implementation

- Two-dimensional table with states x transitions

```
enum states {NO_ALARM, ALARM} current_state;  
enum events {SMOKE_DETECTED, NO_SMOKE_DETECTED} new_event;  
typedef void (*transition)();
```

```
transition state_table[2][2] = {  
    {alarm, no_alarm},          // state NO_ALARM  
    {alarm, no_alarm} };       // state ALARM
```

Each element in `state_table` is a function that handles the corresponding input event. It must be implemented and produce the correct output and the `current_state` must be adapted accordingly.

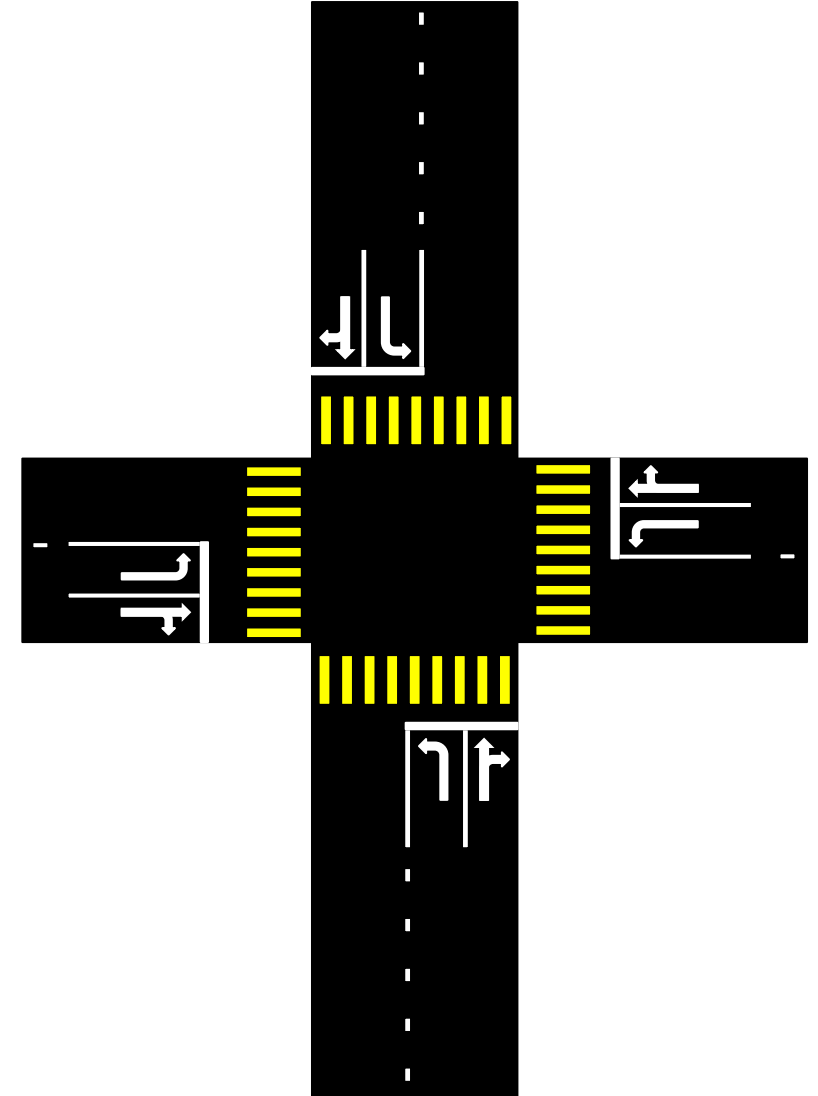
Possible Issues with FSMs

- To describe a complex system, we need to describe many states and many transitions
→ State space explosion

 - In simple State Machines, there is no notion of concurrency, hierarchy, and communication
→ See next week for the extensions used in UML State Machines to cope with these restrictions
- Improvements on FSMs have been proposed by D. Harel in 1984:
<http://www.wisdom.weizmann.ac.il/~dharel/SCANNED.PAPERS/Statecharts.pdf>

Modelling the Traffic Light Control System Software?

- Do we already know everything we need to start the System Modelling?
- “Divide & Conquer”
→ Partitioning System in Sub-Systems
 - Instead of a crossroad traffic light system, could we look at a single traffic light?
 - What would be the consequences?
 - How would our SW model look like?

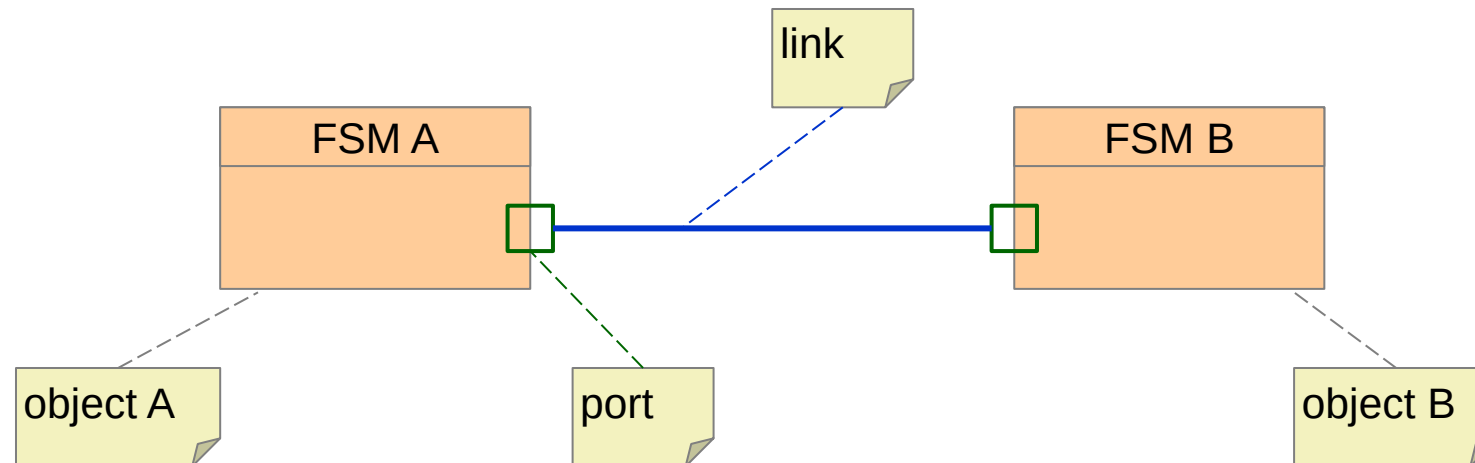


■ Port

- Defines the messages that can be sent and received by an FSM
 - Output message → action of the FSM
 - Input message → event of the FSM

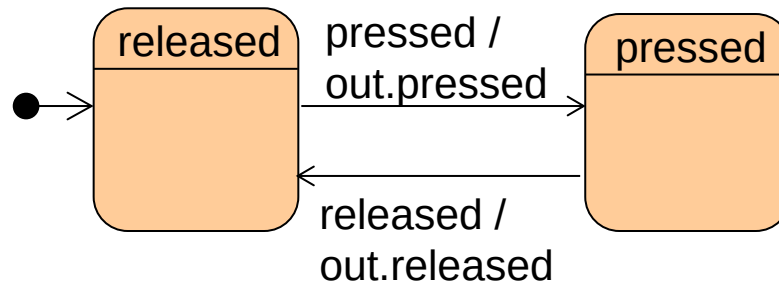
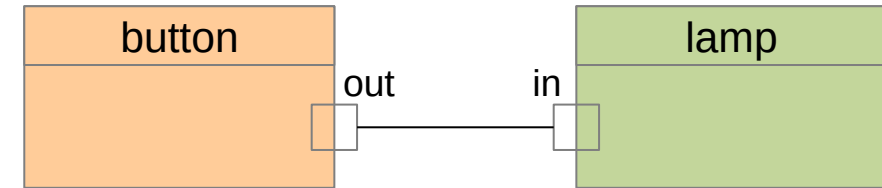
■ Link

- Defines a connection for sending messages

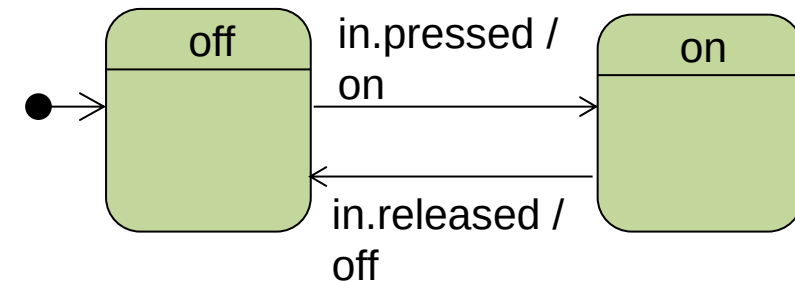


■ Example

- Reactive system partitioned into two FSMs
- Interaction of FSMs happens through event-messages



button produces messages on port out



lamp reacts to events on port in

Conclusions

- **Requirements play an essential role in Embedded System design**
- **RFC2119 defines keywords for standardized requirements writing**
- **Requirements still offer many options to developers to take their own decisions concerning system design**
- **UML State Machines are well suited to model software for Embedded Systems**